German University in Cairo (GUC)

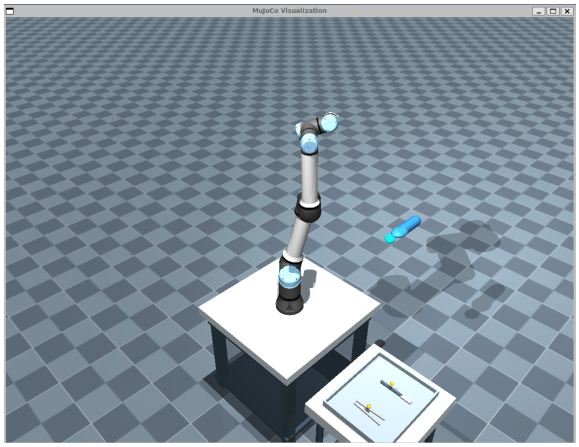
Faculty of Engineering & Materials Science
Mechatronics Engineering Department

MCTR911 – Robotics Programming | Winter 2026

Milestone 02

UR5e Robot Kinematics, ROS 2–MuJoCo Integration, and Motion Visualization

Application: Simulated Autonomous Surgical Tool Handover



Team members and IDs: Abdelrahman Emad 58-2133
Basmala Yasser 58-2375
Omar Khattab 58-24063
Samer Ramzy 58-19523
Hebatalla Mohamed 58-2372
Mariam Ayman 58-13210

Date 13 October 2026

Abstract

This report documents the second milestone of an MCTR911 Robotics Programming project using a six-degree-of-freedom UR5e manipulator for a simulated surgical instrument handover application. The work covers assignment of Denavit–Hartenberg (DH) frames, standard DH parameters, analytical forward position kinematics, and numerical inverse position kinematics using the Jacobian transpose. The kinematic calculations are implemented in Python and compared with the UR5e MuJoCo model. Five forward-kinematics configurations and three inverse-position-kinematics targets passed the reported static pose validation checks. The robot is connected to MuJoCo through a ROS 2 bridge; Python nodes provide fixed targets, step, ramp and sinusoidal commands, and six interactive joint sliders. An initial workstation containing a dedicated robot table, a surgical instrument table, a tray, instrument placeholders, and a handover marker has been implemented. This milestone establishes the kinematics and simulation interface, but does not yet implement autonomous grasping, collision-free handover trajectories, or human tracking.

Contents

1 Robot Model and Coordinate Frames **2**

2 Denavit–Hartenberg Convention **3**

3 Forward Position Kinematics **3**

4 Inverse Position Kinematics **4**

5 ROS 2–MuJoCo Simulation Integration **5**

6 Joint Actuation, Input Signals, and Motion Visualization **6**

7 Surgical Handover Workstation and GUI **7**

8 Validation Results and Discussion **8**

9 Reproducibility and Submission Materials **9**

10 Conclusion **10**

1 Robot Model and Coordinate Frames

1.1 UR5e Manipulator and Simulated Tool Frame

The selected manipulator is the UR5e with six revolute joints. The model is stored in `Milestone01/RobotModel/ur5e.xml`; `scene.xml` includes that model into a MuJoCo MJCF scene. In joint order, the articulations are `shoulder_pan_joint`, `shoulder_lift_joint`, `elbow_joint`, `wrist_1_joint`, `wrist_2_joint`, and `wrist_3_joint`. The end-effector reference used here is the named MuJoCo `attachment_site`. All angles are measured in radians and lengths in metres.

The DH base frame $\{0\}$ is chosen to coincide with the MuJoCo world frame at the robot base. This matters because the kinematics functions return the tool pose directly in the same reference frame as `data.site_xpos` and `data.site_xmat`. Other world objects may be added without changing this mapping, provided the robot's own base and joint hierarchy remain fixed.

1.2 Frame Assignment

Following Tutorial 03 [**?**], the z-axis associated with a revolute joint is aligned with that joint's rotation axis. The common-normal direction determines the corresponding x-axis. Only x- and z-axes need to be drawn: $y_i = z_i \times x_i$ follows from the right-hand rule. Under *standard* DH indexing, z_{i-1} represents joint i's rotation axis. For the terminal frame, z_6 is selected parallel to z_5, and the frame is placed at the tool attachment site.

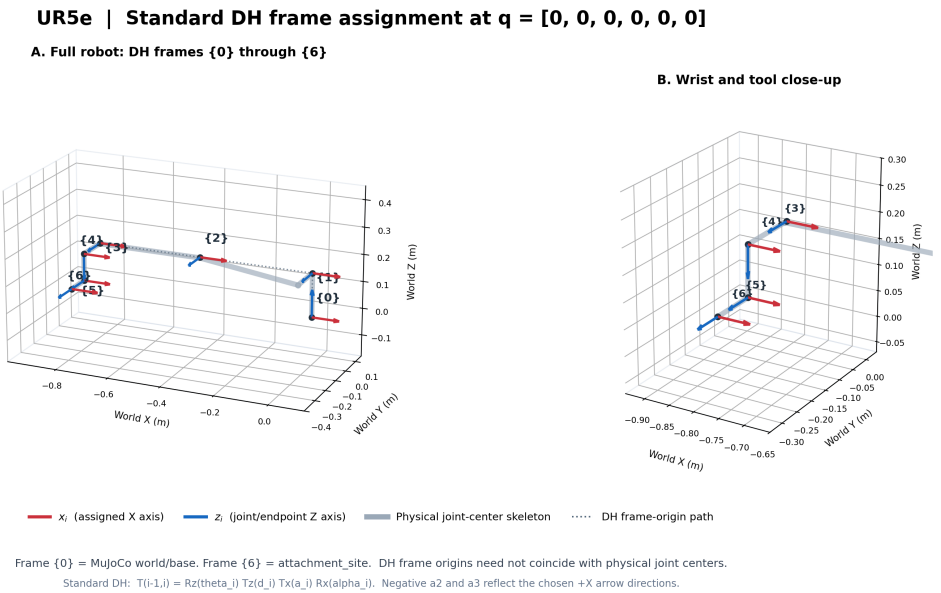


Figure 1: Assigned standard-DH x_i (red) and z_i (blue) axes for the zero-joint configuration. The physical joint-center skeleton and chosen DH frame-origin path are shown separately; the origins need not coincide.

The x_i directions in Table 1 are not chosen to make all link lengths positive. Signed DH distances are valid and preserve the orientation of the selected frames. In particular, the shoulder–elbow links point along negative world X in the zero configuration while the chosen DH x-axes point along positive X.

Table 1: DH frame origins and axes in the MuJoCo world frame at $\boldsymbol{q} = 0$. Axes listed as world-coordinate directions.

Frame	**Origin** (x, y, z) **[m]**	z_i	x_i
0	(0, 0, 0)	$+Z$	$+X$
1	(0, 0, 0.163)	$-Y$	$+X$
2	(−0.425, 0, 0.163)	$-Y$	$+X$
3	(−0.817, 0, 0.163)	$-Y$	$+X$
4	(−0.817, −0.134, 0.163)	$-Z$	$+X$
5	(−0.817, −0.134, 0.063)	$-Y$	$+X$
6	(−0.817, −0.234, 0.063)	$-Y$	$+X$

2 Denavit–Hartenberg Convention

The standard DH transform ${}^{i-1}T_i$ is formed by the ordered operations $R_z(\theta_i)$, $T_z(d_i)$, $T_x(a_i)$, and $R_x(\alpha_i)$, exactly as in Tutorial 03 [**?**]:

$$ {}^{i-1}T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{1} $$

Here θ_i is rotation about z_{i-1}, d_i is displacement along z_{i-1}, a_i is displacement along x_i, and α_i is the twist around x_i.

Table 2: Standard DH parameters for the actual simulated UR5e model. The joint variable is $q_i = \theta_i$.

Joint i	θ_i	d_i **[m]**	a_i **[m]**	α_i **[rad]**
1	q_1	0.163	0	$+\pi/2$
2	q_2	0	-0.425	0
3	q_3	0	-0.392	0
4	q_4	0.134	0	$+\pi/2$
5	q_5	0.100	0	$-\pi/2$
6	q_6	0.100	0	0

The 0.163, 0.425, and 0.392 m dimensions correspond to link offsets encoded in the team's MJCF robot hierarchy. The wrist displacement $d_4 = 0.134\,\text{m}$ results from combining the relevant signed body offsets $0.138 - 0.131 + 0.127 = 0.134\,\text{m}$. Because this is a particular DH frame assignment, numerical parameters should not be mixed with a different UR5e convention without also transforming the frames and applying any required joint-angle offsets.

3 Forward Position Kinematics

3.1 Transformation Chain

Let $\boldsymbol{q} = [q_1, \ldots, q_6]^T$. The complete tool pose relative to the world/base frame is

$$ {}^0T_6(\boldsymbol{q}) = {}^0T_1(q_1)\,{}^1T_2(q_2)\,{}^2T_3(q_3)\,{}^3T_4(q_4)\,{}^4T_5(q_5)\,{}^5T_6(q_6) = \begin{bmatrix} R(\boldsymbol{q}) & \boldsymbol{p}(\boldsymbol{q}) \\ \mathbf{0}^T & 1 \end{bmatrix}. \tag{2} $$

The first three entries in the final column give the Cartesian attachment-site position $\boldsymbol{p} = [x, y, z]^T$, while the 3×3 upper-left block gives its orientation. For the present MJCF model, the selected frame $\{6\}$ coincides with `attachment_site`; an additional tool-offset matrix is not necessary.

The Python file `ur5e_dh_fk.py` stores the six fixed tuples (a_i, α_i, d_i) and multiplies Equation (1) over the supplied joint-angle vector. The code validates the input shape and returns a 4×4 NumPy array.

3.2 Worked Reference Configurations

At $\boldsymbol{q} = \mathbf{0}$, the calculated transformation is

$$ {}^{0}T_{6}(\mathbf{0}) = \begin{bmatrix} 1 & 0 & 0 & -0.817 \\ 0 & 0 & -1 & -0.234 \\ 0 & 1 & 0 & 0.063 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{3} $$

Therefore $\boldsymbol{p}_0 = (-0.817, -0.234, 0.063)^T$ m. At the MJCF home configuration,

$$ \boldsymbol{q}_{\text{home}} = (-1.5708, -1.5708, 1.5708, -1.5708, -1.5708, 0)^T \text{ rad}, \tag{4} $$

the calculated attachment-site position is approximately $\boldsymbol{p}_{\text{home}} = (-0.133998, 0.491999, 0.488000)^T$ m. Both positions were checked against MuJoCo using identical joint angles. The full-pose validation procedure and quantitative errors are given in Section 8.

4 Inverse Position Kinematics

4.1 Problem Statement and Choice of Numerical Method

Inverse position kinematics determines $\boldsymbol{q}$ for a desired Cartesian point $\boldsymbol{p}_d$:

$$ \boldsymbol{p}(\boldsymbol{q}) = \boldsymbol{p}_d, \qquad \boldsymbol{e}(\boldsymbol{q}) = \boldsymbol{p}_d - \boldsymbol{p}(\boldsymbol{q}). \tag{5} $$

The course's Tutorial 04 presents geometric methods and numerical root finding, including Newton–Raphson, for inverse kinematics [**?**]. For this six-joint robot the *position-only* Jacobian is 3×6; it has no ordinary matrix inverse. The team's solver uses the **Jacobian transpose** method, an iterative numerical alternative. It should not be described as an exact Newton–Raphson inverse-Jacobian update.

4.2 Position Jacobian from the DH Frames

For revolute joint i, let $\boldsymbol{z}_{i-1}$ denote its unit axis direction, $\boldsymbol{o}_{i-1}$ its DH-frame origin, and $\boldsymbol{p}$ the end-effector position, all expressed in frame $\{0\}$. Then

$$ \boldsymbol{J}_{v,i} = \boldsymbol{z}_{i-1} \times (\boldsymbol{p} - \boldsymbol{o}_{i-1}), \qquad J_v = \begin{bmatrix} \boldsymbol{J}_{v,1} & \cdots & \boldsymbol{J}_{v,6} \end{bmatrix} \in \mathbb{R}^{3\times 6}. \tag{6} $$

The code obtains $\boldsymbol{z}_{i-1}$ and $\boldsymbol{o}_{i-1}$ from the intermediate DH transformations. Thus the IK procedure uses the same frame convention as forward kinematics and the frame diagram.

4.3 Jacobian Transpose Iterations

The update rule is

$$ \boxed{\boldsymbol{q}_{k+1} = \boldsymbol{q}_k + \eta J_v(\boldsymbol{q}_k)^T \left[\boldsymbol{p}_d - \boldsymbol{p}(\boldsymbol{q}_k)\right]}, \tag{7} $$

where $\eta > 0$ is the numerical gain. The implemented solver uses a default gain of 1.0, a position tolerance of 10^{-5} m, and a maximum of 3000 iterations. It also limits the magnitude of each proposed joint update to 0.20 rad, clips candidate angles to configured approximate joint bounds, and performs backtracking (trial fractions $1, 1/2, \ldots, 1/64$) if necessary to reduce Cartesian error. These safeguards improve robustness without computing any Jacobian inverse or pseudoinverse.

1. Supply $\boldsymbol{p}_d$ and a feasible seed $\boldsymbol{q}_0$.
2. Evaluate $\boldsymbol{p}(\boldsymbol{q}_k)$ and $J_v(\boldsymbol{q}_k)$ from the DH chain.
3. If $\|\boldsymbol{e}\|_2 \leq 10^{-5}$ m, return the current solution.
4. Compute the Jacobian-transpose correction; apply step limiting, joint-bound clipping and a reducing backtracking step if required.
5. Repeat; otherwise report failure or non-convergence rather than returning an unverified solution.

This solves *end-effector position only*. The tool orientation is not constrained, joint solutions are not necessarily unique, and the method does not produce a collision-free trajectory. It may converge slowly or stall near singular, poorly conditioned or unreachable targets. Multiple initial seeds may produce distinct valid solutions.

5 ROS 2–MuJoCo Simulation Integration

5.1 Software and Communication Architecture

The software setup uses Ubuntu 24.04 under WSL2, ROS 2 Jazzy, MuJoCo 3.2.0, the `mujoco_ros2` bridge, and Python `rclpy` nodes [**?**]. The bridge loads the MJCF scene, applies joint-position commands, and publishes the current measured joint state. The communications flow is shown in Figure 2.

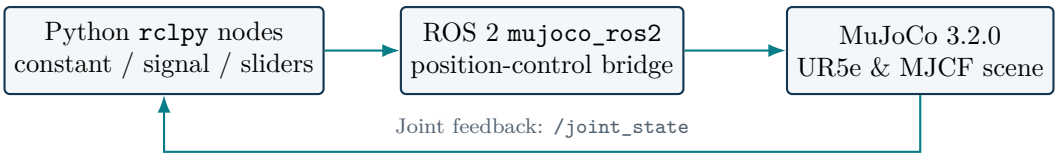


Figure 2: Joint-command and feedback loop implemented using ROS 2 Python nodes and MuJoCo.

The command publisher uses `std_msgs/msg/Float64MultiArray` to send a six-element target vector in actuator order. The subscriber receives `sensor_msgs/msg/JointState` on `/joint_state` and reads the six reported positions. The basic controller in `ur5e_controller.py` starts from the first observed configuration, adds +0.3 rad to J1, and republishes the target at 10 Hz. The other controllers preserve all non-selected joints at their initially observed values.

5.2 Launch Configuration and Renderer

The scene used for the application is `Milestone 01/Robot Model/scene_handover.xml`, with the original `ur5e.xml` preserved beside it. The bridge uses `control_mode=POSITION` and the topic parameters above. For the final local development setup, `simulation_frequency=250` Hz and `visualisation_frequency=1` Hz were selected after investigating slow WSLg rendering. The application was launched with Mesa's D3D12 driver targeting an NVIDIA GeForce GTX 1660 Ti. These frequencies are *requested configuration values*; measured ROS topic throughput is discussed separately in Section 8.4. Previously captured controller CSV files may have been acquired using other visualization settings and must not be relabeled without checking their run records.

Listing 1: Core bridge launch commands (from the repository root).

```
source /opt/ros/jazzy/setup.bash
source ~/mctr911_ws/install/setup.bash
SCENE="$PWD/Milestone 01/Robot Model/scene_handover.xml"
env -u LIBGL_ALWAYS_SOFTWARE -u LIBGL_ALWAYS_INDIRECT \
    GALLIUM_DRIVER=d3d12 \
```

```
MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA \
ros2 run mujoco_ros2 mujoco_node "$SCENE" \
--ros-args -p control_mode:=POSITION \
-p joint_state_topic_name:=joint_state \
-p joint_command_topic_name:=joint_commands \
-p simulation_frequency:=250 \
-p visualisation_frequency:=1
```

6 Joint Actuation, Input Signals, and Motion Visualization

6.1 Input Modes and Python Programs

The milestone asks for motion analysis using constants, signal builders, and sliders. This is achieved through separate Python ROS 2 nodes; only one command publisher is run at a time to avoid conflicting targets. The initial joint vector observed from feedback is $\boldsymbol{q}_{\text{ref}}$. For a selected joint j, the signal generator publishes

$$\boldsymbol{q}_d(t) = \boldsymbol{q}_{\text{ref}} + u(t)\boldsymbol{e}_j, \tag{8}$$

where $\boldsymbol{e}_j$ is a unit vector selecting joint j. The implemented input modes are described in Table 3.

Table 3: Actuation methods used in Milestone 02. Angles are in radians.

Mode	Desired joint offset $u(t)$	Implementation
Constant	$+0.20$ from initial J1 position	`constant_controller.py`; 10 Hz publisher
Step	0 for $t < 1\,\text{s}$; $+0.20$ afterward (example)	`ur5e_signal_generator.py`
Ramp	$0.20\,\min(t/25, 1)$ (example)	Same generator; 25 s ramp time
Sine	$0.25\sin(2\pi \cdot 0.01t)$ (example)	Same generator; 0.01 Hz frequency
Sliders	Each joint: user-specified $[-0.30, 0.30]$ offset	`slider_controller.py` (Tkinter, six sliders)

The signal-generator examples were run with `-joint 1` and a command publication rate of 10 Hz. The slider interface also publishes at 10 Hz and displays the desired and actual joint angles. Its *Hold current pose / Recenter sliders* control sets a new reference from the latest feedback; slider inputs are relative offsets, not global actuator-limit sliders.

6.2 Response Logging and Visualization

The scripts save time-stamped CSV records in `Milestone 02/Figures/`. Signal and slider recordings include desired and actual joint angles; signal logs also include tracking errors for all six joints. `plot_joint_response.py` produces (a) the desired and actual angle trajectories and (b) the tracking error $e_j(t) = q_{d,j}(t) - q_{a,j}(t)$. Its optional summaries include

$$\text{MAE} = \frac{1}{N}\sum_{k=1}^{N} |e_j(t_k)|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_j(t_k)^2}. \tag{9}$$

No aggregate MAE or RMSE values are asserted here because the underlying log files have not been reprocessed as part of this report-generation pass.

7 Surgical Handover Workstation and GUI

7.1 Application-Oriented Scene Layout

The new MJCF scene `scene_handover.xml` reuses `ur5e.xml` without changing the robot's base transform or joint hierarchy. Instead, the floor was moved to $z = -0.65\,\text{m}$ and a dedicated robot-support table was constructed with its tabletop surface at $z = 0$. As required by the instructor, the manipulator is therefore visibly mounted on *its own table*. A second table contains a shallow tray with simplified scalpel and forceps geometry. A symbolic surgeon-hand object and a cyan `handover_target_site` identify the proposed transfer location.

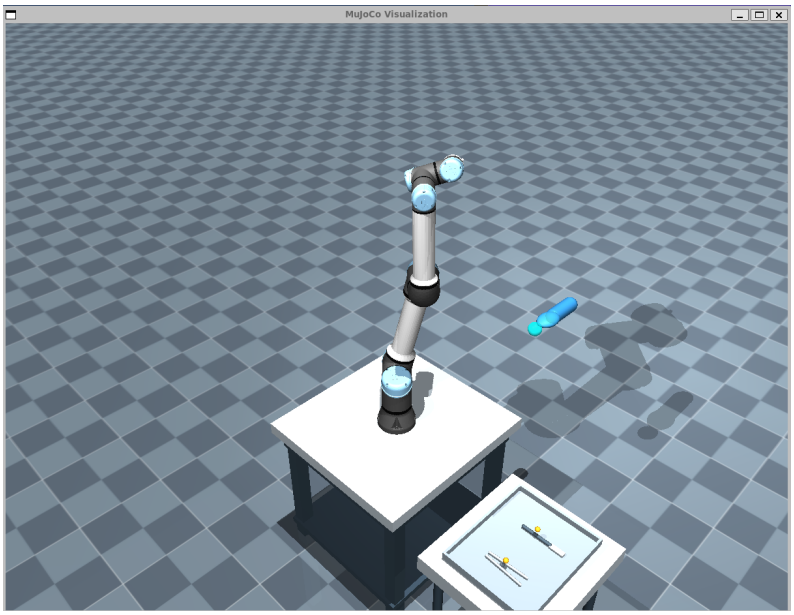


Figure 3: MuJoCo visualization of the initial surgical handover workstation, showing the separate robot-support and instrument tables, tray, static tool models and symbolic handover marker. Screenshot captured during Milestone 02 development.

Table 4: Representative world-coordinate placements in `scene_handover.xml`.

Component	World position [m]	Role
Robot base	$(0, 0, 0)$	Fixed original kinematic base
Floor	$z = -0.65$	Lowered world floor
Robot tabletop	Top at $z = 0$	Dedicated mounting surface
Instrument table	$(0.75, -0.20, 0.05)$	Separate tool workstation
Scalpel center	$(0.75, -0.13, 0.099)$	Simplified visual instrument
Forceps center	$(0.75, -0.29, 0.099)$	Simplified visual instrument
Handover marker	$(0.30, 0.43, 0.38)$	Fixed transfer target

7.2 Scope of the Environment

The new workstation components are intentionally *static, visual-only* objects for Milestone 02; their new geoms disable collision interactions. The surgical tools cannot yet be physically grasped and transferred. The handover site is not a dynamically tracked human hand. MuJoCo provides a 3D visual simulation environment while the separate Tkinter slider window serves as the user-facing interactive joint-control GUI. Both represent the *start* of the application GUI rather than a complete autonomous surgical procedure.

8 Validation Results and Discussion

8.1 Independent Forward-Kinematics Verification

The test `validate_kinematics.py` loads the MuJoCo MJCF, assigns identical joint angles by joint name, calls `mujoco.mj_forward` and compares `attachment_site` against the independent DH implementation. This checks static kinematics, *not* physics-driven tracking. Position error is $\|\boldsymbol{p}_{\mathrm{DH}} - \boldsymbol{p}_{\mathrm{MJ}}\|_2$; orientation error is the angular distance between the rotation matrices.

Table 5: Observed forward-kinematics validation results from five tested configurations.

Configuration	Position error [m]	Orientation error [deg]	Result
Zero	2.408×10^{-16}	3.508×10^{-15}	PASS
Home	1.777×10^{-16}	1.349×10^{-14}	PASS
Configuration A	1.665×10^{-16}	5.147×10^{-15}	PASS
Configuration B	2.776×10^{-17}	1.151×10^{-14}	PASS
Configuration C	2.436×10^{-16}	1.159×10^{-14}	PASS
All five static FK tests			**PASS**

The largest observed positional difference was 2.408×10^{-16} m, and the largest angular difference was 1.349×10^{-14} degrees. These differences are at numerical floating-point precision. The tests therefore strongly support the chosen DH frame assignment and transformation chain for the particular supplied robot model.

8.2 Inverse Position Kinematics Verification

The `validate_ik_mujoco.py` test first generates target XYZ positions from MuJoCo at reference configurations. The Jacobian-transpose solver then computes joint angles from a supplied seed; these joint angles are assigned back to MuJoCo, which independently returns the reached XYZ position. The pass tolerance is 10^{-5} m.

Table 6: MuJoCo-verified inverse-position-kinematics tests (Jacobian transpose).

Target	Iterations	MuJoCo XYZ error [m]	Result
Test A	106	9.275×10^{-6}	PASS
Test B	252	9.715×10^{-6}	PASS
Home target	92	9.831×10^{-6}	PASS
All three static position-IK tests			**PASS**

The three tests converged to within 0.01 mm of the target position. The observed iterations (92–252) are higher than would typically be required by some inverse-based methods, but the transpose update is straightforward to derive and does not require inversion. Successful position IK does not establish a matching tool orientation; the terminal wrist may assume a different pose from the seed's corresponding reference configuration.

8.3 Preservation of Kinematics After Scene Expansion

The team checked both `scene.xml` and `scene_handover.xml` using the same six joint angles and `mj_forward`. The world-frame position and rotation matrix of `attachment_site` matched exactly (as printed by the comparison script) in the zero and home configurations:

This establishes that the workstation additions did not alter the tested kinematics. The model also

Table 7: Original versus expanded scene: direct attachment-site comparison.

Pose	Position difference [m]	Rotation-matrix difference (Frobenius norm)
Zero	0.000e+00	0.000e+00
Home	0.000e+00	0.000e+00

compiled successfully with six generalized velocities (six DoF), six actuators, and 14 bodies. The robot was subsequently moved interactively from Python sliders in the expanded scene.

8.4 Simulation-Rate Observation and Limitations

During WSL2 development, OpenGL initially reported software rendering (`llvmpipe`). Explicit selection of Mesa's D3D12 driver enabled hardware acceleration on the NVIDIA GTX 1660 Ti. At the requested physics frequency of 250 Hz, the observed `/joint_state` average rate was about 1.136 Hz with visualization at 20 Hz, increasing to about 115 Hz when visualization was reduced to 1 Hz. These are observed ROS topic message rates, not direct measurements of simulation physical time or robot motor bandwidth. They demonstrate a practical dependence of the current bridge workflow on visualization load. The final development launcher uses 250 Hz physics and 1 Hz visualization; a higher visual rate may be selected for smoother screen recording.

Important limitations at this stage are as follows: (i) validation with `mj_forward` tests static geometry rather than the motor-controller transient response; (ii) the numerical IK method constrains position, not orientation; (iii) collisions with new workstation props are disabled; (iv) simulated grasp/release and autonomous tool delivery are not implemented; and (v) motion-response observations may differ under alternative simulator and rendering settings. These features are to be addressed in later milestones rather than claimed as completed in Milestone 02.

9 Reproducibility and Submission Materials

9.1 Code and File Organization

Listing 2: Milestone 02 source and evidence structure (simplified).

```
Milestone 02/
  kinematics/
    ur5e_dh_fk.py
    ur5e_ik_transpose.py
    plot_ur5e_dh_frames.py
    validate_kinematics.py
    validate_ik_mujoco.py
    ur5e_dh_frames.png
  Code/ROS2/
    ur5e_controller.py
    constant_controller.py
    ur5e_signal_generator.py
    slider_controller.py
    plot_joint_response.py
  Figures/
    *.csv
    Plots/*.png
  Report/
    report.tex                # this report
    figures/                  # report's embedded figures
  Videos/                     # team video: still to attach
Milestone 01/Robot Model/
  ur5e.xml
```

```
  scene.xml
  scene_handover.xml
  assets/
```

9.2 Reproduce the Kinematic Tests

From `Milestone 02/kinematics` with the team's Python kinematics virtual environment active:

```
python ur5e_dh_fk.py
python validate_kinematics.py
python ur5e_ik_transpose.py
python validate_ik_mujoco.py
# Optional: test the expanded scene explicitly
python validate_kinematics.py --scene \
  "../../Milestone 01/Robot Model/scene_handover.xml"
python validate_ik_mujoco.py --scene \
  "../../Milestone 01/Robot Model/scene_handover.xml"
```

9.3 Reproduce ROS 2 Motion Experiments

Open one terminal for the MuJoCo bridge, using Listing 1. In a second terminal, source ROS 2 Jazzy and work from the team repository root. Execute one controller at a time:

```
source /opt/ros/jazzy/setup.bash
python3 "Milestone 02/Code/ROS2/constant_controller.py"
python3 "Milestone 02/Code/ROS2/ur5e_signal_generator.py" \
  --mode step --joint 1 --amplitude 0.20 \
  --duration 40 --rate 10
python3 "Milestone 02/Code/ROS2/ur5e_signal_generator.py" \
  --mode ramp --joint 1 --amplitude 0.20 \
  --ramp-time 25 --duration 50 --rate 10
python3 "Milestone 02/Code/ROS2/ur5e_signal_generator.py" \
  --mode sine --joint 1 --amplitude 0.25 \
  --frequency 0.01 --duration 100 --rate 10
python3 "Milestone 02/Code/ROS2/slider_controller.py"
```

The programs publish to the same ROS 2 command topic, so they should never be run simultaneously. The constant controller overwrites its fixed output CSV filename if rerun; other controllers use time-stamped filenames.

10 Conclusion

Milestone 02 established a consistent standard-DH model for the UR5e and implemented its forward and Jacobian-transpose inverse *position* kinematics in Python. Static MuJoCo validation passed five full-pose forward-kinematics checks and three position-only inverse-kinematics tests. ROS 2 Python nodes successfully commanded the UR5e through the MuJoCo bridge using constants, generated signals and a six-slider interface. The expanded simulator environment represents the intended surgical handover application with a dedicated robot table, separate instrument table, simplified tools and a marked handover point. Remaining tasks for future milestones include motion planning, tool orientation and grasping, and controlled execution of the complete handover procedure.